

## Lab Exercise: Hands on Ride-Hailing Datasets – with parquet file FROM BASICS to ML Modeling

### Submission Instructions

- Submit a **Single Jupyter Notebook** (.ipynb) containing:
  - All code cells with outputs.
  - Answers to all questions as markdown cells.
- Ensure the notebook runs from start to finish without errors.
- **Deadline:** 12<sup>th</sup> May 2026

This lab is designed to reinforce the statistical concepts covered in Lab 1 (descriptive statistics, variable types, central tendency, dispersion, skewness, visualizations) using the ride-hailing datasets (**synthetic, ~10 million records, NYC dataset consists of over 3 million records**). You will practice extracting actionable insights and preparing data for machine learning. At last, you need to conclude what differences you identified working between real vs synthetic datasets.

---

### Dataset Description

The dataset contains trip records with the following columns:

| Column Name       | Type      | Description                             |
|-------------------|-----------|-----------------------------------------|
| trip_id           | String    | Unique identifier for the trip          |
| rider_id          | String    | Anonymized rider ID                     |
| driver_id         | String    | Anonymized driver ID                    |
| trip_start_time   | Timestamp | Start of trip (YYYY-MM-DD HH:MM:SS)     |
| trip_end_time     | Timestamp | End of trip                             |
| pickup_lat        | Float     | Pickup latitude                         |
| pickup_lon        | Float     | Pickup longitude                        |
| dropoff_lat       | Float     | Dropoff latitude                        |
| dropoff_lon       | Float     | Dropoff longitude                       |
| trip_distance     | Float     | Distance in kilometers                  |
| trip_duration     | Integer   | Duration in seconds                     |
| fare_amount       | Float     | Total fare paid by rider                |
| surge_multiplier  | Float     | Surge pricing multiplier (1.0 = normal) |
| driver_rating     | Float     | Rider's rating of driver (1-5)          |
| rider_rating      | Float     | Driver's rating of rider (1-5)          |
| payment_method    | String    | Cash / Card / Wallet                    |
| traffic_condition | String    | Light / Moderate / Heavy                |
| weather           | String    | Clear / Rain / Fog / Storm              |

|                    |         |                                      |
|--------------------|---------|--------------------------------------|
| day_of_week        | Integer | 0=Monday ... 6=Sunday                |
| hour_of_day        | Integer | 0-23                                 |
| driver_tenure_days | Integer | How many days driver has been active |
| rider_tenure_days  | Integer | How many days rider has been active  |

The full dataset (10 million rows) is available in Parquet format: `swiftride_10M.parquet`.  
Use **fastparquet** engine to avoid compatibility issues.

---

## Lab Tasks

### Part 0: Setup

#### # All of the key imports

*# Load the Parquet file (adjust path)*

```
df = pd.read_parquet('swiftride_10M.parquet', engine='fastparquet')
```

#### Questions:

1. Print the shape of the dataset and column names.
  2. Check for missing values. If any, decide how to handle them (drop, fill, etc.) and explain why.
- 

### Part 1: Variable Classification

For each column, classify as:

- **Categorical** → nominal (no order) or ordinal (order matters)
- **Numerical** → discrete (countable) or continuous (measurable)

Create a table in markdown with your answers.

#### Example start:

| Column        | Type        | Subtype    | Justification                     |
|---------------|-------------|------------|-----------------------------------|
| trip_id       | Categorical | Nominal    | Unique identifier, no order       |
| trip_distance | Numerical   | Continuous | Can take any floating-point value |

| Column      | Type        | Subtype | Justification                       |
|-------------|-------------|---------|-------------------------------------|
| day_of_week | Categorical | Ordinal | 0=Monday ... 6=Sunday implies order |

---

## Part 2: Summary Statistics

`df.describe()`

### Questions:

1. What is the average fare\_amount? What is its standard deviation?
  2. Which numerical column has the largest range? Why might that be?
  3. Look at the min and max of driver\_rating. Are there any unrealistic values? If yes, propose a cleaning step.
- 

## Part 3: Measures of Central Tendency & Dispersion

Compute the following **using Pandas functions**:

- Mean, median, and mode of trip\_distance
- Range, variance, and standard deviation of fare\_amount
- Interquartile range (IQR) of trip\_duration

### Questions:

1. Is the distribution of trip\_distance symmetric or skewed? Justify by comparing mean and median.
  2. Which measure of central tendency is most appropriate for driver\_rating? Why?
- 

## Part 4: Categorical Variable Analysis

For weather and traffic\_condition columns:

```
print(df['weather'].value_counts(normalize=True))

print(df['traffic_condition'].value_counts(normalize=True))
```

### Questions:

1. What is the most common weather condition? The least common?
  2. Create a **bar chart** showing trip counts by traffic\_condition.
  3. What percentage of trips occurred during Heavy traffic?
- 

## Part 5: Group Comparisons

Answer the following by writing Pandas code:

1. Average driver\_rating for weather = 'Storm' vs. weather = 'Clear'.  
(Hint: group by weather and compute mean rating)
2. Average fare\_amount for payment\_method = 'Cash' vs. 'Card'.
3. For trips with surge\_multiplier > 1.5, what is the average trip\_distance?
4. Calculate the **standard deviation** of trip\_duration for weekend trips (day\_of\_week ≥ 5) and weekday trips.
5. Calculate the **skewness** of fare\_amount for trips with driver\_rating ≥ 4.5.
6. **Mean difference** of trip\_distance between traffic\_condition = 'Heavy' and 'Light'.

Use .groupby(), .mean(), .std(), .skew(), etc.

---

## Part 6: Derived Columns & Binning

Create three new columns:

```
df['duration_min'] = df['trip_duration'] / 60                    # continuous
df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)         # binary
df['driver_exp'] = pd.cut(df['driver_tenure_days'],
                           bins=[0,30,90,180,10000],
                           labels=['New (<30d)', 'Moderate (30-90d)',
                                    'Experienced (90-180d)', 'Veteran (>180d)'])
```

### Questions:

1. What is the average duration\_min for driver\_exp = 'New (<30d)'?
2. How many is\_weekend = 1 trips have surge\_multiplier > 2.0?

---

## Part 7: Visualizations

Create the following plots using matplotlib and seaborn. Include proper titles, labels, and legends.

1. **Scatter plot** – trip\_distance (x) vs fare\_amount (y), colored by surge\_multiplier.  
*What pattern do you observe?*
2. **Box plot** – driver\_rating by weather.  
*Which weather condition has the lowest median rating? The highest spread?*
3. **Line plot** – Average driver\_rating per hour\_of\_day.  
*At what hours are ratings lowest? Suggest a reason.*
4. **Histogram** – Distribution of driver\_tenure\_days (use log scale on y-axis).  
*Why does this plot look heavily skewed toward low tenure?*

---

## Part 8: Extract Actionable Insights

Based on the analyses above, write **ten insights** in the following format:

**Insight 1:** [Observation], **Evidence:** [Specific numbers/statistics], **Recommendation:** [What the business should do]

**Example** (from the lecture):

**Insight:** New drivers receive much lower ratings during storm weather compared to experienced drivers.

**Evidence:** New drivers (tenure <30 days) average rating in storm = 3.1, experienced (>90 days) = 4.6.

**Recommendation:** Implement mandatory weather-driving simulation training for all new drivers.

---

## Part 9: Prepare Dataset for Machine Learning

Drop ID columns and any columns that would not be available at prediction time (e.g., trip\_end\_time if predicting before trip ends). Then save the cleaned dataset as a **Parquet** file.

```
df_model = df.drop(['trip_id', 'rider_id', 'driver_id', 'trip_start_time', 'trip_end_time'], axis=1)
```

```
df_model.to_parquet('swiftride_cleaned.parquet', engine='fastparquet', index=False)
```

**Question:** Why is Parquet a better choice than CSV for saving this cleaned dataset? What difference you noticed?

## Part-2 Yellow Taxi

| Column Name           | Type      | Description                                                                               |
|-----------------------|-----------|-------------------------------------------------------------------------------------------|
| VendorID              | Integer   | Code indicating the TPEP provider (1=Creative Mobile, 2=VeriFone)                         |
| tpep_pickup_datetime  | Timestamp | Date and time when the meter was engaged                                                  |
| tpep_dropoff_datetime | Timestamp | Date and time when the meter was disengaged                                               |
| passenger_count       | Float     | Number of passengers in the vehicle (driver-entered)                                      |
| trip_distance         | Float     | Trip distance in miles                                                                    |
| RatecodeID            | Float     | Rate code (1=standard, 2=JFK, 3=Newark, 4=Nassau/Westchester, 5=negotiated, 6=group ride) |
| store_and_fwd_flag    | String    | Trip record held in vehicle memory before sending to vendor (Y/N)                         |
| PULocationID          | Integer   | TLC Taxi Zone where the trip started                                                      |
| DOLocationID          | Integer   | TLC Taxi Zone where the trip ended                                                        |
| payment_type          | Integer   | 1=Credit card, 2=Cash, 3=No charge, 4=Dispute, 5=Unknown                                  |
| fare_amount           | Float     | Time-and-distance fare                                                                    |
| extra                 | Float     | Extra charges (e.g., rush hour, night)                                                    |
| mta_tax               | Float     | \$0.50 MTA tax                                                                            |
| tip_amount            | Float     | Tip amount (automatically populated for credit card payments)                             |
| tolls_amount          | Float     | Total tolls paid                                                                          |
| improvement_surcharge | Float     | \$1.00 improvement surcharge (2015 onwards)                                               |
| total_amount          | Float     | Total charged to passengers (sum of all amounts)                                          |
| congestion_surcharge  | Float     | \$2.50 congestion surcharge (2019 onwards)                                                |
| Airport_fee           | Float     | \$1.25 airport fee for trips originating from LaGuardia or JFK                            |
| cbd_congestion_fee    | Float     | Central Business District congestion fee (new in 2026)                                    |

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

# Load the Parquet file (adjust path)

df = pd.read\_parquet('yellow\_tripdata\_2026-01.parquet', engine='fastparquet')

1. Print the shape of the dataset and list all column names.
2. Check for missing values. Which columns have missing values and how many? Suggest a handling strategy (e.g., drop, fill with median) and justify.
3. Display the first 5 rows and last 5 rows.
4. Print the data types of each column.

For **each** column (or at least 10 key columns), classify as:

- **Categorical** → nominal (no order) or ordinal (order matters)
- **Numerical** → discrete (countable) or continuous (measurable)

Create a markdown table with columns: Column, Type, Subtype, Justification.

| Column        | Type        | Subtype    | Justification                                                       |
|---------------|-------------|------------|---------------------------------------------------------------------|
| VendorID      | Categorical | Nominal    | Two possible values (1,2), no inherent order                        |
| trip_distance | Numerical   | Continuous | Can take any positive real number                                   |
| payment_type  | Categorical | Nominal    | Codes represent different payment methods, but 1≠2 in numeric sense |

1. What is the **mean** and **standard deviation** of fare\_amount?
2. Which numerical column has the largest **range** (max – min)? Why might that be?
3. Look at the min and max of trip\_distance. Are there unrealistic values? Propose a reasonable cap.

Compute the following using Pandas functions:

- Mean, median, and mode of trip\_distance
- Range, variance, and standard deviation of fare\_amount
- Interquartile range (IQR) of tip\_amount

#### Questions:

1. Is the distribution of trip\_distance symmetric or skewed? Justify by comparing mean and median.
2. Which measure of central tendency is most appropriate for passenger\_count? Why?
3. Why might tip\_amount have a large IQR?
4. Print the 25th, 50th, and 75th percentiles of total\_amount.
5. What is the most common payment type? What percentage of trips use it?
6. Create a **bar chart** showing the count of trips by payment\_type.
7. What does store\_and\_fwd\_flag = 'Y' mean? What percentage of trips have this flag?
8. Create a **pie chart** for store\_and\_fwd\_flag.
9. Average fare\_amount for payment\_type = 1 (Credit card) vs. 2 (Cash).
10. Average tip\_amount for credit card vs. cash trips.
11. For trips with passenger\_count = 1, what is the average trip\_distance?
12. Calculate the **standard deviation** of trip\_distance for trips with VendorID = 1 and VendorID = 2.
13. Calculate the **skewness** of fare\_amount for trips with tip\_amount > 0.
14. **Mean difference** of total\_amount between trips that paid Airport\_fee (>0) and those that did not.
15. What is the average trip\_duration\_min for weekend trips vs. weekday trips?

16. What is the average tip\_percentage for credit card payments?
17. Create a distance\_bin column using pd.cut() with bins: [0, 1, 3, 5, 10, 100] and labels ['Very Short', 'Short', 'Medium', 'Long', 'Very Long'].
18. Count the number of trips in each distance\_bin.

Create the following plots using matplotlib and seaborn. Include proper titles, labels, and legends.

1. **Scatter plot** – trip\_distance (x) vs fare\_amount (y), colored by payment\_type. *What pattern do you observe?*
2. **Box plot** – total\_amount by payment\_type. *Which payment type has the highest median total amount? Which has the most outliers?*
3. **Line plot** – Average tip\_percentage per pickup\_hour. *At what hours are tips highest? Suggest a reason.*
4. **Histogram** – Distribution of trip\_distance (cap at 20 miles for clarity). Use log scale on y-axis. *Why is the distribution highly skewed?*
5. **Heatmap** – Correlation matrix of numerical columns:  
(fare\_amount, trip\_distance, tip\_amount, trip\_duration\_min, passenger\_count). *Which pair has the strongest correlation? What does that imply?*

### Lastly, extract Actionable Insights

Based on your analyses, write **five insights** in the following format:

**Insight 1:** [Observation]

**Evidence:** [Specific numbers/statistics]

**Recommendation:** [What the business should do]

TLC Trip Record Data - TLC <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

After completing the given lab material, you are suggested to take a look at the following links and cover the contents:

<https://www.kaggle.com/code/ghulamhiader/hypothesis-testing>

<https://www.kaggle.com/code/vvineeth/maximizing-revenue-hypothesis-testing/input>